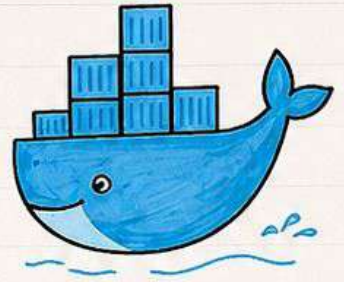


DOCKER

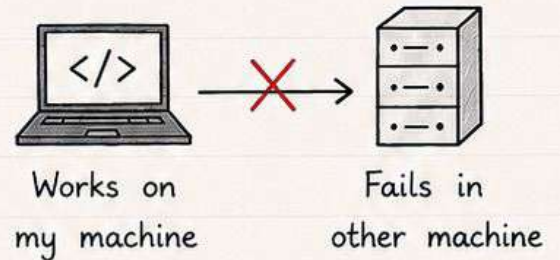


What is Docker?

Docker is an open-source platform that helps us build, ship and run applications inside **containers**.

Why was Docker Created?

To solve the “**it works on my machine**” problem. Docker makes sure the app runs same in every environment.



Why Docker is Useful?

- **Portability** : Run anywhere (dev, test, prod)
- **Lightweight** : Containers use fewer resources
- **Fast Deployment** : Build, test and deploy quickly
- **Isolation** : Each container runs in its own environment
- **Consistency** : Same environment everywhere

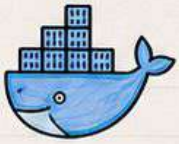
Write once,
Run
anywhere !

IMPORTANT CONCEPTS

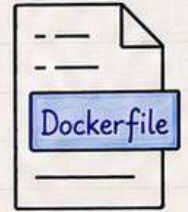
-  **Dockerfile** – A text file that contains instructions to build a Docker image.
-  **Image** – A read-only template used to create containers.
-  **Layers** – An image is made up of multiple read-only layers.
-  **Registry** – A place to store and share Docker images (e.g., Docker Hub).
-  **Engine** – The core part of Docker that builds, runs and manages containers.
-  **Container** – A running instance of a Docker image.



Remember : Image is the **blueprint**, Container is the running app.

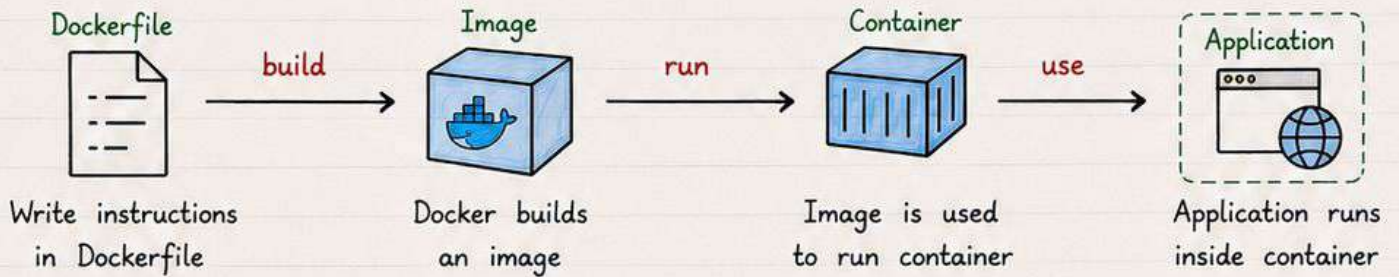


DOCKERFILE



A Dockerfile is a **text file** that contains a set of instructions to build a Docker image automatically.

How Dockerfile Works ?



Common Instructions in Dockerfile

FROM	Sets the base image.
WORKDIR	Sets the working directory inside the container.
COPY	Copies files or folders from host to container.
RUN	Executes commands while building the image.
EXPOSE	Informs Docker that the container listens on the specified port.
CMD	Specifies the default command to run when the container starts.

Example Dockerfile

```
1 FROM node:20          # Base image
2 WORKDIR /app           # Set working directory
3 COPY package*.json ./  # Copy dependency files
4 RUN npm install        # Install dependencies
5 COPY . .               # Copy all files
6 EXPOSE 3000            # Expose port
7 CMD ["npm", "start"]   # Start the application
```

Explanation

- Uses official Node.js 20 image as base.
- Sets /app as the working directory.
- Copies package.json to install deps first (better caching).
- Installs all dependencies defined in package.json.
- Copies the rest of the application code.
- Container will listen on port 3000.
- Runs 'npm start' when container starts.

Build and Run

• Build Image

```
$ docker build -t myapp .
```

↑
• (dot) means current directory

• Run Container

```
$ docker run -p 3000:3000 myapp
```

↑ ↑
Host port Container port



Dockerfile makes image creation repeatable, portable and easy.

★ What is a Docker Image?

- An image is a **read-only** template with instructions for creating a container.
- It contains application code, runtime, libraries, environment variables and files.
- Images are **immutable**.

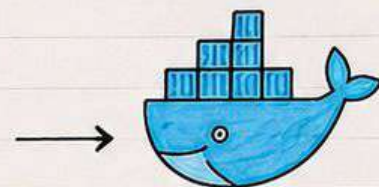
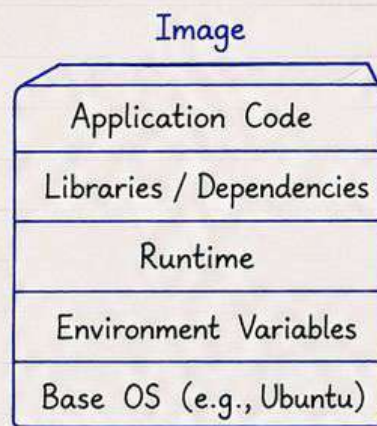
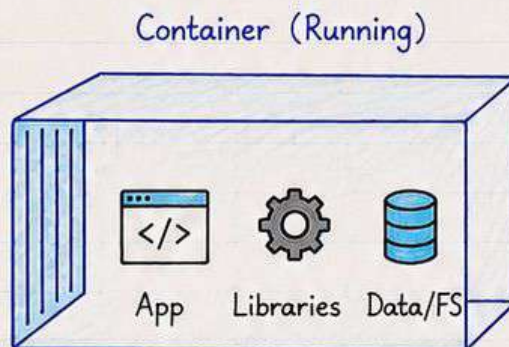


Image is built from multiple layers.

★ What is a Docker Container?

- A container is a **running instance** of an image.
- It has its own filesystem, network, process space but shares the host OS kernel.
- Containers are **ephemeral** (can be created, started, stopped and removed easily).

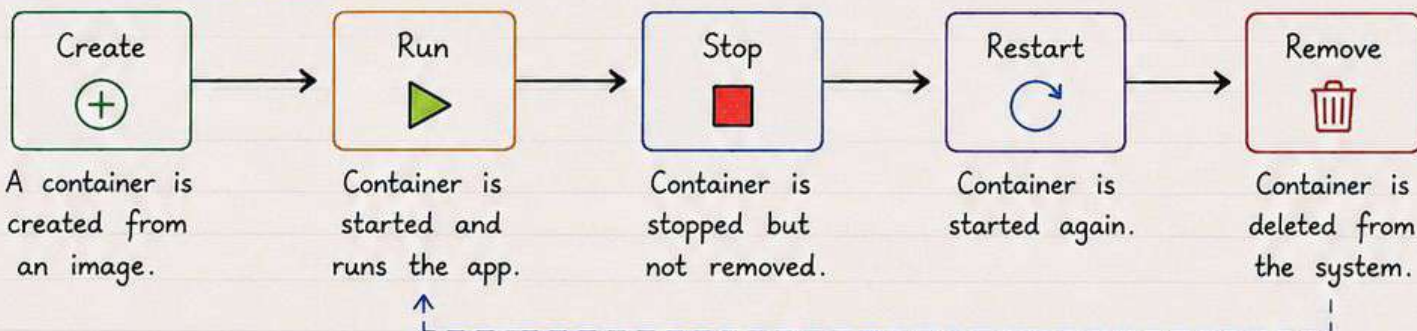


- Isolated environment
- Lightweight
- Portable
- Consistent

Image vs Container

Docker Image	Docker Container
• Read-only template	• Running instance of an image
• Created using Dockerfile	• Created from an image
• Immutable	• Mutable (changes are temporary)
• Stored in Docker Hub or local	• Exists on Docker engine (local)
• Does not have a state	• Has a state (running/stopped)
• Can be shared and reused	• Cannot be shared

Container Lifecycle



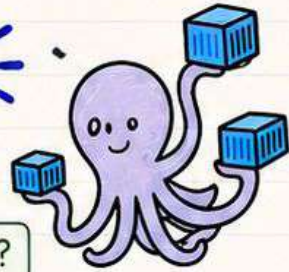
★ Key Points

- ✓ Image = Blueprint
- ✓ Container = Running process

- ✓ Image is built once, container can be created multiple times.
- ✓ Containers are fast, lightweight and portable.



≡ DOCKER COMPOSE ≡



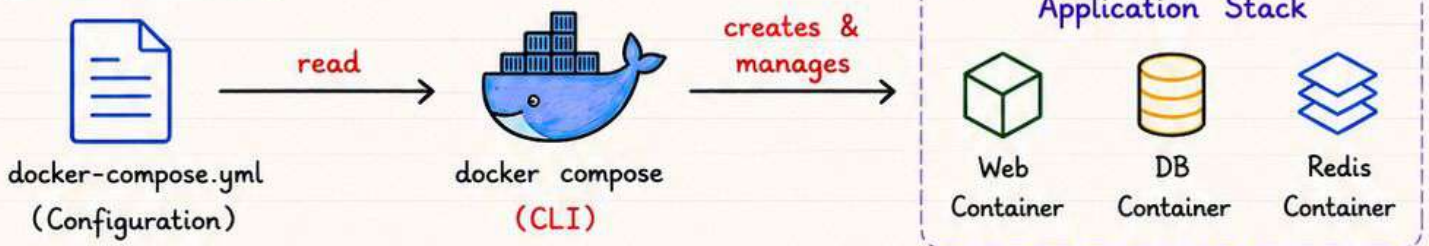
What is Docker Compose ?

Docker Compose is a tool for **defining** and **running** multi-container Docker applications. We define our app's **services**, **networks** and **volumes** in a **docker-compose.yml** file and **orchestrate** everything with simple commands.

Why use Docker Compose ?

- Manage multi-container apps easily
- Single configuration file
- Start, stop and manage with one command
- Consistent environments for everyone

How it Works ?



Example : docker-compose.yml

```
version: '3.8'
services:
  web:
    image: nginx:latest
    ports:
      - "80:80"
    depends_on:
      - db
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: root
    volumes:
      - db_data:/var/lib/mysql
volumes:
  db_data:
```

version : Version of Compose file format

services : List of containers (services) in the app

web / db : Each service definition

- **image** - Docker image to use
- **ports** - Expose container ports
- **depends_on** - Start db before web
- **environment** - Set environment variables
- **volumes** - Persist data

Common Commands

- **docker compose up** → Start all services
- **docker compose up -d** → Start in detached mode (background)
- **docker compose down** → Stop and remove containers, networks
- **docker compose ps** → List running services
- **docker compose logs** → View logs of services
- **docker compose build** → Build or rebuild services



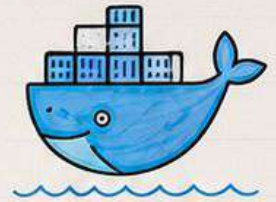
One docker-compose.yml file can manage an entire application stack !



Remember : Compose is great for development, testing and small deployments.

For large scale production, use orchestration tools like **Docker Swarm** or **Kubernetes**.

5 IMPORTANT DOCKER COMMANDS



These 5 commands are enough to get you started with Docker!

1 docker build

Build an image from a Dockerfile and tag it with a name.

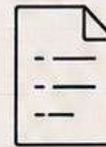
Syntax :

```
docker build -t <image-name> <path>
```

Example :

```
docker build -t myapp .
```

Dockerfile



build

Image



myapp

2 docker run

Create and start a container from an image.

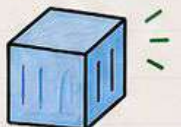
Syntax :

```
docker run -d -p <host-port>:<container-port> <image-name>
```

Example :

```
docker run -d -p 8080:80 nginx
```

Container



(running)

3 docker ps

List all running containers.

Syntax :

```
docker ps
```

Example :

```
docker ps
```

Running Containers

CONTAINER ID	IMAGE	STATUS	PORTS
a1b2c3d4e5f6	nginx	Up 5 min	0.0.0.0:8080->80/tcp
-----	----	----	-----

4 docker stop

Stop a running container.

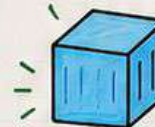
Syntax :

```
docker stop <container-id / name>
```

Example :

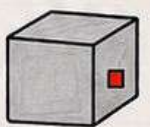
```
docker stop myapp
```

Running



stop

Stopped



5 docker rm

Remove a stopped container from the system.

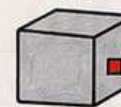
Syntax :

```
docker rm <container-id / name>
```

Example :

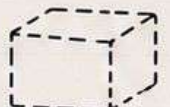
```
docker rm myapp
```

Stopped



Stopped Container

remove



Removed



Remember :

- Use **-d** to run container in **detached** (background) mode.
- Use **-p** to publish (map) ports.

